Lctr 2: Hello from UART

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Fundamentals of UART (universal asynchronous receiver and transmitter).
- Parameters and basic calculations for UART.

Demo project:

- UART setup in CubeMX and parameters.
- HAL (hardware abstract layer) functions for UART.
- `stdout` and `stdin` redirection to UART.
- Requirements of the project.
- Project management in CubeIDE.
- Running the demo project using Renode.

2.1 Introduction

2.1.1 UART and MCU

A modern MCU usually has multiple USARTs (universal synchronous and asynchronous receiver and transmitter), as illustrated in Fig. 1. A UART is a simplified version of USART without functionalities related to synchronized communications; details will be provided later.

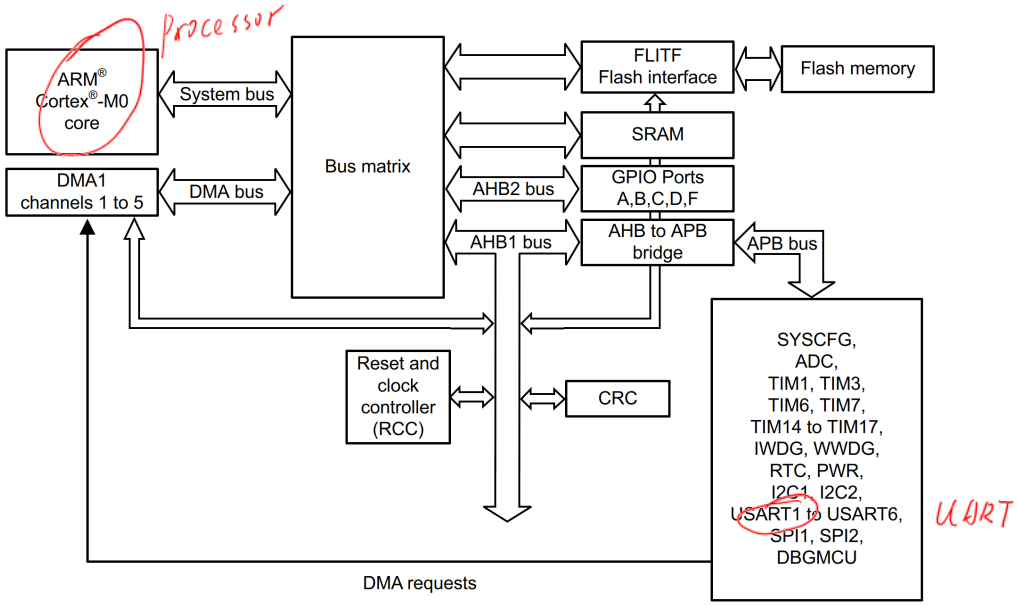


Figure 1: STM32H7A3 and STM32H7B3 MCUs with different pin counts and memory sizes.

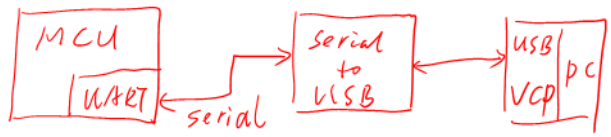
To program a UART for data communications, we also need to use the following hardware, in addition to the MP core and UART:

- The **reset and clock controller (RCC)**. For a UART to work, we need to provide it with a clock from RCC. At this stage, we can just use the default setups of clocks.
- Flash and SRAM. Flash is used to save program code, and SRAM (static random access memory) is used to save program variables.

2.1.2 UART is THE choice for MCU communications

- Even the simplest MCU will have at least one UART.
- All PCs can support serial communications. Modern PCs use virtual serial com port (VCP). With VCP, a user application program can talk to an MCU in the *language* of UART.

2.2 Principle of UART



2.2.1 Meaning of USART

Now, let's take a look at the meaning of the major terms of USART:

- **Universal** means the device is programmable to suit for different needs.
- **Transmitter** and **receiver** refer to the hardware circuitry used to support two-way communications; combined together, they form a **transceiver**.
- **Synchronous** vs **asynchronous**:
  - Synchronous communication transceivers use a common clock wire to synchronize the transmitting and receiving devices, as shown in Fig. 2.
  - Asynchronous communication transceivers use predetermined clocks at both devices. Here, start and stop bits have to be used to synchronize the frames, as shown in Fig. 3. The transmitter and the receiver should use the same transmission speed and data frame structure.

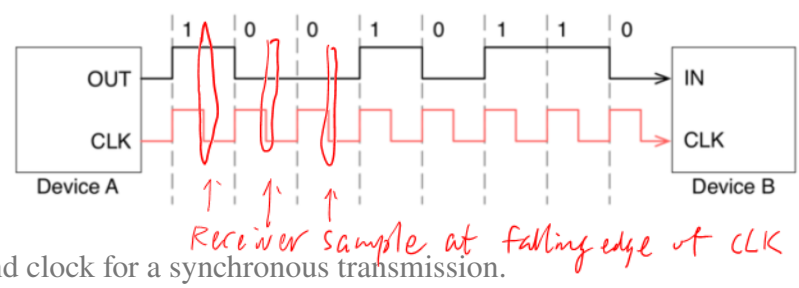


Figure 2: Data and clock for a synchronous transmission.

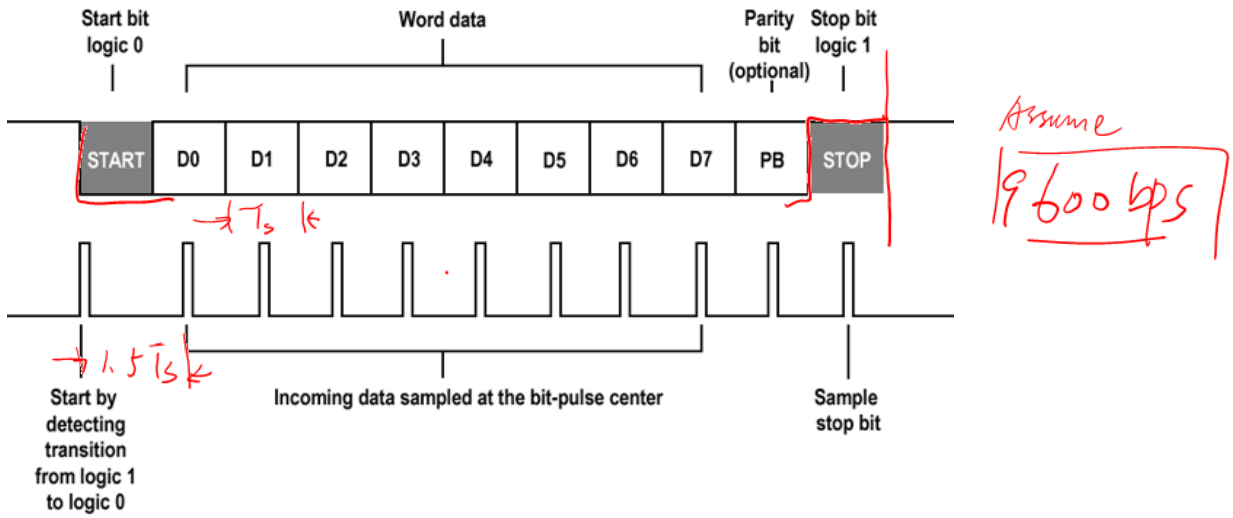


Figure 3: Data and start and stop bits for an asynchronous transmission.

Except the ground wire, we use three and two wires for USART and UART, respectively, as shown in Fig. 4.

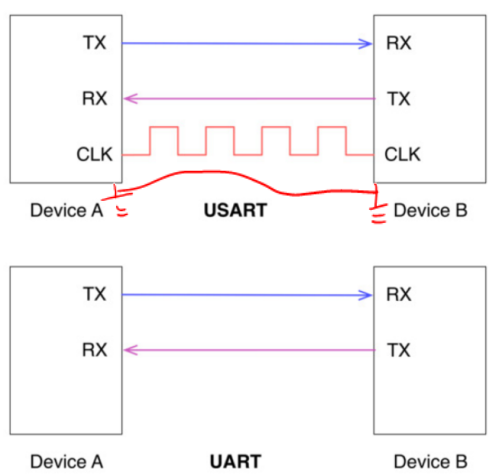


Figure 4: Connection illustration of USART and UART communications.

We will only use the term UART in the sequel to simplify the presentation.

2.2.2 Structure of a data frame

Here are the basic ideas leading to the structure of a data frame for serial communications:

- We use a certain clock rate to sample the data at the receiver to determine if the level is high or low, corresponding to logic 1 and 0, respectively.
- To synchronize the sampling with transmission, we need to know when to sample. This is done by using the following design:
  - Idle level should be high.
  - One start bit, which is low, will signal the sampling of bit 0.
  - Data bits are sent one by one, followed by an *optional* parity bit. The following should be agreed by both the transmit and receive parties before communications:
    - Raw data (without parity check bit) size. It can be 6, 7, and 8 bits per frame.
    - Transmission sequence. Data can be LSB (least significant bit) first or MSB (most significant bit) first. Use LSB first by default.
  - There are one or two stop bits. The choice has to be agreed before communications start as well.

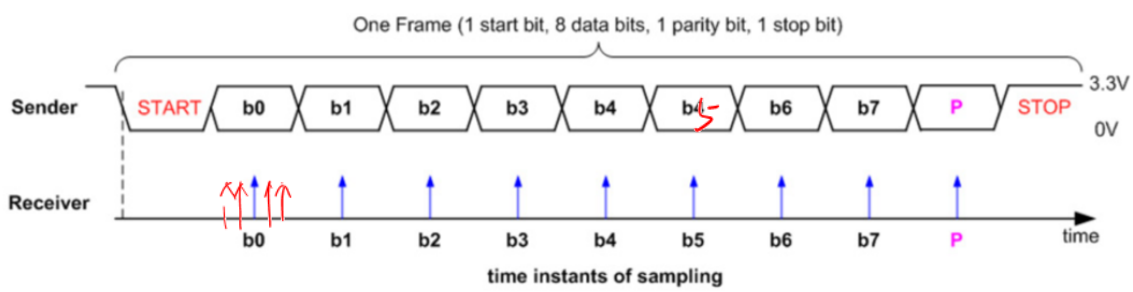
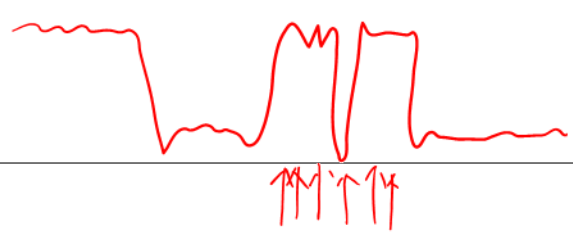


Figure 5: Data frame for an 8 bit plus 1 parity transmission.

Note that the stop bit is always high and the start bit is always low so they can work together nicely.

Modern implementation of UART often uses oversampling to average out the noise to reduce communication errors.



oversampling will be able to address this issue

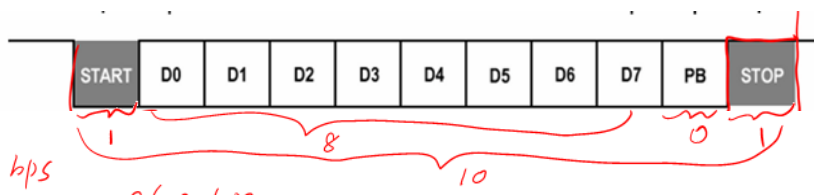
2.2.3 Baud rate

In UART, baud rate is just the rate of clock used to send the data, which is not the same as the definitions for other application cases:

- Historically, baud rate is used in telecommunication to represent the number of pulses physically transferred per second.
- In digital communication, baud rate is the number of bits physically transferred per second.

Example 1: Consider a case of using a baud rate of 9600 bps (bits per second) with the following data frame: one start bit, 8 data bits, one stop bit, and no parity bit. What is the transmission rate of the **actual** data in terms of Bytes per second?

Solution:



① Bits per frame = 10.

② Given bit rate 9600 bps

$\Rightarrow$ Frame rate = Byte rate = $\frac{9600 \text{ bps}}{10 \text{ b/frame}}$ = 960 frames/s = $\boxed{960 \text{ Bytes/s.}}$

Note that the start, stop, and parity bits are the *protocol overhead*.

2.2.4 Error detection

UART can use a single parity bit to detect one bit error in a frame.

P

- Even parity: total number of ones in the transmission is even.
- Odd parity: total number of ones in the transmission is odd.

Example of load word (data + parity) of 5 is 1010_0000

0 for even parity.
1 for odd parity

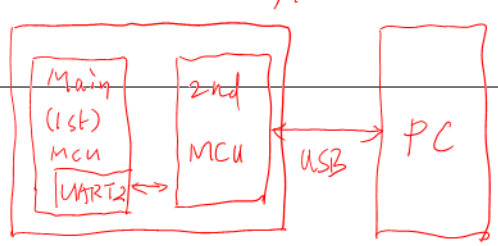
Example 2: What will be the parity bit if we use Even Parity to transmit a 8-bit binary sequence 0b0111_1110?

Note that we use **underscores** to segment bits into groups of 4 bits for better readability.

Solution:

① There are 6 ones in the load data

② P = 0 since we have even ones already.



Three functions of this USB:
① Serial/USB converter
② Debugger
③ Storage

2.3 Demo project: `stdout` and `stdin` redirection to UART

We use UART2 for redirection since this is the UART connected to the serial to USB converter on all the STM32 MPBs we use.

2.3.1 UART2 setups for serial communications

We have discussed before that we use CubeMX for the setup of MCUs. Specifically, we need to do the following:

- Initializing the UART hardware.
- Setting up the UART parameters, including: baud rate, number of bits in a frame, parity check, and number of stop bits.
- Generating the toolchain project; it is a CubeIDE project for our case.

It is too early for us to discuss the details of the above operations now; those who are curious can see the details in the Setting up USART2 for VCP and Generating code from CubeMX sections of the A MiniRef for STM32CubeMX chapter of the Companion.

Here, we use a preconfigured project, `cc1s_uart_redirect`, which has the following setups for UART2:

- **Baud Rate**: *115200 Bits/s*
- **Word Length**: *8 Bits (including Parity)*
- **Parity**: *None*
- **Stop Bits**: *1*
- **Data Direction**: *Receive and Transmit*.
- **Over Sampling**: *16 Samples*.
- **Single Sample** (if this appears): *Disable*.

2.3.2 HAL functions for receiving and transmitting via UART

We use the following two HAL (hardware abstract layer) functions:

- For transmission:

```
HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart, const uint8_t *pData, uint16_t Size, uint32_t Timeout)
```

- For receiving:

```
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout)
```

2.3.3 `stdout` and `stdin` redirection to UART

The `printf` and `scanf` functions are designed in such a way that we can specify what device we use for the physical printout and read.

For GNU compilers, we need to implement the following two `stdout` and `stdin` redirection functions using the above HAL UART functions:

```
int __io_putchar(int ch) {
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
    return ch;
}

int __io_getchar(void) {
    uint8_t ch = 0;
    __HAL_UART_CLEAR_OREFLAG(&huart2);
    HAL_UART_Receive(&huart2, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
    return ch;
}
```

For details of redirect, including for different toolchains, please see the code in `uart_redirect.c` and `uart_redirect.h` located in the `lib` folder of the demo project.

Note that `huart2` is the handle for UART2, which is **a pointer to the data structure** used for operating UART2 created by CubeMX. We need to include the `mp_supported_mcu.h` file in the project so that this handle can be used in the project.

Note also that in `uart_redirect.h`, we define the following macro

```
#define SET_STDIN_TO_NO_BUFFER setvbuf(stdin, NULL, _IONBF, 0)
```

which is used to address some issues of the CubeMX generated code, as detailed in https://forum.digikey.com/t/easily-use-scanf-on-stm32/21103. We need to use this micro to set up the `stdin` buffer before we use `scanf` in all our projects to avoid unexpected issues. This is illustrated below.

2.3.4 Requirements of the project

The main purpose of the demo project is to illustrate how to program UART on the MCU so that it can communicate with a PC via a terminal tool, such as PuTTY.

Like most of the demo projects, the demo project of this lecture, `cc1s_uart_redirect`, can run on both Renode via F412dsc and a real MPB, G431n32.

Here are what the executable of this project does:

- Print out "Hello World" on Renode's UART terminal or PC's VCP terminal.
- Remind the user to input a string from the terminal.
- Print back the string on the terminal.

To get a smooth start, we will only use Renode here.

2.3.5 Preparing the project files

Download the enclosed `cc1s_uart_redirect.zip` file and unzip it to the `proj_mp` folder. All files of this demo project, including the Renode script file to run the executable, are there. You should have the following folder structure:

```
proj_mp
  cc1s_uart_redirect
    cc1s_uart_redirect_f412dsc
```

```
cc1s_uart_redirect_g431n32
lib
renode
src
```

2.3.6 Project management in CubeIDE

In CubeIDE, we need to do the following:

- Embedding the hook in the CubeMX generated code.
- Organizing the user code.
- Building of the project.

2.3.6.1 Adding a hook in the generated code

The code generated by CubeMX will do necessary hardware initializations, including that for UART. After that, it will go to a `while (1)` loop, as shown below, a code snippet from `main.c` located at `cc1s_uart_redirect\cc1s_uart_redirect_f412dsc\Src`:

```c
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_USART2_UART_Init();
/* USER CODE BEGIN 2 */

/* USER CODE END 2 */

/* Infinite loop */
while (1) {
  /* USER CODE BEGIN 3 */

  /* USER CODE END 3 */
}
```

Here, we add a hook function for user code using the following two lines in the space for `USER CODE 2`:

```c
void mp_main(void);
mp_main();
```

This way, we can write all our user code in another source file, which can be used by all the MPBs

that can be used by this project.

// To cover on 1/12/26

2.3.6.2 Putting all user code in user source files

We put all our user code related to this project to the `src` folder. For this specific project, we have two source files. The **first** one, `_mp_main.c`, is the same for all projects used in this class. It is the implementation of the hook function, used to connect the generated code to the real user logic code, as shown below:

```
void mp_main(void) {
#if defined(UNIT_TEST)
    void mp_unity(void);
    mp_unity();
#else
    void mp_app(void);
    mp_app();
#endif
}
```

For this project, we don't have unit test, and the entrance to the code for our App is `mp_app`, which is defined in the **second** source file, `cc1s_uart_redirect_app.c`, which has all our user logic code, as shown below:

```
#include <stdio.h>
#include "mp_uart_redirect.h"

void mp_app(void) {
    printf("Hello World!\n");

    SET_STDIN_TO_NO_BUFFER;
    char str[50];
    while (1) {
        printf("Input a string: \n");
        scanf("%s", str);
        printf("The string you have just input is %s.\n", str);
    }
}
```

In the above code list, we do the following:

1. Print out "Hello World!".
2. Set the `stdin` buffer with `SET_STDIN_TO_NO_BUFFER`, as discussed above.
3. In the `while (1)` loop, we repeat the process of getting an input string from the user and then echo it back.

2.3.6.3 Linking the source files and including the header files

These are already set up in the provided project. We will detail how to do that in later project assignments. If you cannot wait, please refer to the corresponding sections of the A MiniRef for STM32CubeIDE chapter of the Companion.

2.3.6.4 Building the project with CubeIDE

Building the project in CubeIDE can be done easily by clicking the **[Build]** button. Note that we only need to build the `Debug` configuration.

2.3.7 Running the program with Renode

See class demo or the detailed notes in Chapter 2 of the Collection.